

Sistemas Distribuidos - SoundColab

Edgardo Dorner¹, Esperanza Tejeda¹, Francisco Labrin¹, and Vicente Alves¹

Universidad Austral de Chile, Valdivia, Chile

`esperanza.tejeda@alumnos.uach.cl`

`edgardo.dorner@alumnos.uach.cl`

`francisco.labrin@alumnos.uach.cl`

`vicente.alves@alumnos.uach.cl`

Resumen El presente trabajo propone el diseño de un sistema distribuido orientado al procesamiento de audio ambiental, cuyo objetivo principal es generar mapas de ruido y clasificar eventos acústicos, incluyendo la detección de sonidos asociados a aves. El sistema recolecta grabaciones de audio desde dispositivos móviles distribuidos geográficamente, las encola para su procesamiento asíncrono, y aplica modelos de clasificación para categorizar las fuentes sonoras. La arquitectura propuesta sigue un estilo por capas con patrón cliente-servidor, utilizando Python con Flask para los servicios REST, Redis como sistema de colas, PostgreSQL como base de datos relacional y Docker para la contenerización de los componentes. Se presenta el diseño de la arquitectura de componentes, el modelo relacional de datos, el modelo físico de despliegue y el modelo fundamental del sistema. Como caso de estudio, el sistema será implementado en el campus Miraflores de la Universidad Austral de Chile.

Keywords: Distributed systems · Audio processing · Acoustic classification · Bird detection · Asynchronous processing · Noise mapping

1. Introducción

El análisis del entorno acústico ha adquirido creciente relevancia en ámbitos como la planificación urbana, el monitoreo ambiental y el estudio de la biodiversidad. En particular, la capacidad de identificar patrones de ruido y clasificar fuentes sonoras (como el tráfico, actividad humana o fauna), permite generar información valiosa para la toma de decisiones, tanto en contextos urbanos como naturales. Sin embargo, el procesamiento de grandes volúmenes de datos de audio provenientes de múltiples ubicaciones plantea desafíos significativos en términos de escalabilidad, latencia y eficiencia computacional. En este contexto, los sistemas distribuidos emergen como una solución adecuada para abordar este tipo de problemáticas. Este enfoque no solo mejora el rendimiento global del sistema, sino que también facilita la tolerancia a fallos y la adaptabilidad frente a entornos dinámicos y heterogéneos.

El presente trabajo propone el diseño de un sistema distribuido orientado al procesamiento de audio ambiental, cuyo objetivo es generar un mapa de ruido y realizar la clasificación de eventos acústicos, incluyendo la detección de sonidos asociados a aves. Como caso de estudio inicial, este sistema será implementado y evaluado en el campus Miraflores de la Universidad Austral de Chile, con el fin de analizar su comportamiento en un entorno real y validar su utilidad en contextos académicos y ambientales.

2. Problemática

El monitoreo del entorno acústico presenta desafíos, donde la naturaleza y la vida cotidiana se complementan generando un caos de información proveniente desde distintos puntos geográficos y distintas fuentes de ruido, convirtiendo el análisis y la clasificación manual en una práctica engorrosa e ineficiente. En particular, la identificación de patrones sonoros, como la detección de contaminación acústica o la presencia de aves, requiere procesar grandes cantidades de datos provenientes de distintas ubicaciones geográficas y de distintos tipos de dispositivos de forma constante e incluso concurrente. En este contexto, el problema central que se aborda en este trabajo es la necesidad de un sistema capaz de recolectar, procesar y analizar datos de audio de manera eficiente y escalable, permitiendo generar un mapa de ruido y realizar la clasificación automática de eventos acústicos en tiempo cercano al real. Para ello, se requiere una solución que distribuya la carga de procesamiento, reduzca la latencia y garantice la disponibilidad del sistema frente a posibles fallos.

3. Solución Propuesta

Se propone un sistema distribuido basado en una arquitectura cliente-servidor multicapa que contempla los siguientes flujos principales:

Recolección: Dispositivos móviles (Android/iOS) capturan audio ambiental junto con metadatos de ubicación GPS y condiciones climáticas. A través de la aplicación, los usuarios registrados envían las grabaciones a una API dedicada, la cual valida, almacena el archivo de audio y encola el trabajo de procesamiento en Redis.

Procesamiento: Un servicio de procesamiento consume las tareas de la cola Redis, extrae características acústicas del audio (nivel de decibeles, frecuencia promedio, duración), clasifica la categoría del sonido (tráfico, naturaleza, actividad humana, aves, etc.) mediante la biblioteca `librosa`, y los modelos preentrenados BirdNET para detección de aves y YAMNet para clasificar todo lo demás (Tráfico, Actividad humana, Música, Naturaleza, Animales, Alarmas, Silencio). Los resultados se almacenan en PostgreSQL.

Visualización: Una PWA pública consulta al backend principal para obtener los datos procesados y los presenta en un mapa de ruido interactivo, permitiendo filtrar por categoría acústica, rango de fechas y ubicación geográfica.

Comunicación entre servicios: La comunicación entre los distintos componentes del sistema se realiza utilizando una combinación de mecanismos síncronos y asíncronos, con el objetivo de garantizar seguridad, escalabilidad y desacoplamiento.

- **Comunicación cliente-servidor (Frontend a Backend):** Se implementa mediante APIs REST sobre HTTP/HTTPS. La autenticación de usuarios se gestiona utilizando JSON Web Tokens (JWT), los cuales se envían en cada solicitud a través del encabezado `Authorization: Bearer <token>`. Esto permite mantener un esquema stateless y seguro para el acceso a los recursos del sistema.
- **Comunicación entre servicios backend:** Para la interacción entre servicios internos (por ejemplo, entre backend-core y collector-api), se utiliza autenticación mediante API Keys enviadas en los encabezados HTTP. Este mecanismo permite asegurar las comunicaciones internas sin necesidad de gestionar sesiones de usuario.
- **Comunicación a nivel de infraestructura (Docker Network):** Todos los servicios del sistema se despliegan en contenedores Docker independientes, correspondientes a cada repositorio del proyecto. Estos contenedores se comunican entre sí mediante una red virtual definida con Docker Network, lo que permite el descubrimiento de servicios por nombre y una comunicación interna segura sin exponer puertos innecesarios al exterior. Esta configuración facilita el aislamiento, la portabilidad y la escalabilidad del sistema, permitiendo desplegar múltiples instancias de servicios según la demanda.
- **Comunicación asíncrona:** El envío de tareas de procesamiento se realiza mediante Redis como sistema de colas de mensajes. El servicio `collector-api` publica tareas en la cola, mientras que el servicio `data-processor` las consume de manera desacoplada, permitiendo escalar horizontalmente el procesamiento de audio.
- **Acceso a base de datos:** Los servicios backend acceden a PostgreSQL mediante conexiones seguras utilizando credenciales internas, sin exposición directa a clientes externos.

4. Diseño

4.1. Lista de Componentes

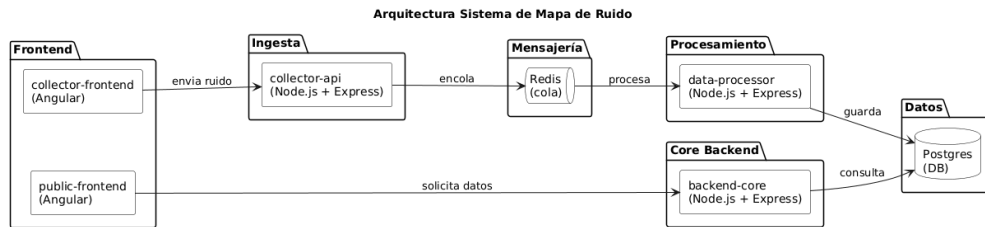


Figura 1. Diagrama de componentes

1. **public-frontend (Angular 17)**: PWA que permite a los usuarios visualizar el mapa de ruido, consultar clasificaciones acústicas y explorar datos por zona geográfica. Se despliega en un contenedor Docker con Nginx como servidor de archivos estáticos. Instalación vía `npm install`.
2. **backend-core (Python 3.11 + Flask 3.0)**: API REST principal que expone los endpoints de consulta de datos procesados para el frontend público. Gestiona la autenticación de usuarios y la lógica de negocio para la visualización. Se despliega en contenedor Docker. Instalación vía `pip install flask`.
3. **collector-frontend (Angular 17)**: PWA orientada a los recolectores de audio. Permite grabar o subir archivos de audio, adjuntar metadatos de ubicación y enviarlos al sistema. Se despliega en contenedor Docker con Nginx.
4. **collector-api (Python 3.11 + Flask 3.0)**: API REST que recibe los archivos de audio desde el collector-frontend, los valida, almacena en el sistema de archivos del servidor y encola las tareas de procesamiento en Redis. Se despliega en contenedor Docker.
5. **data-processor (Python 3.11)**: Servicio trabajador (worker) que consume tareas de la cola Redis, procesa los archivos de audio extrayendo características acústicas con `librosa`, clasifica los eventos sonoros, y almacena los resultados en PostgreSQL. Se despliega en contenedor Docker y puede escalar horizontalmente con múltiples instancias.
6. **PostgreSQL 16**: Base de datos relacional que almacena usuarios, dispositivos, registros de audio, clasificaciones y logs del sistema. Se instala directamente sobre el servidor (sin contenedor, según requerimiento del curso). Instalación vía `apt install postgresql-16`.
7. **Redis 7.2**: Sistema de colas en memoria utilizado como broker de mensajes entre la collector-api y el data-processor. Permite desacoplar la recepción de audio del procesamiento. Instalación vía contenedor Docker: `docker pull redis:7.2`.
8. **Nginx 1.25**: Servidor proxy inverso que actúa como punto de entrada único, enruta las peticiones al frontend o backend correspondiente. Se despliega en contenedor Docker.

4.2. Modelo Relacional

La Figura 2 muestra el modelo relacional de la base de datos del sistema.

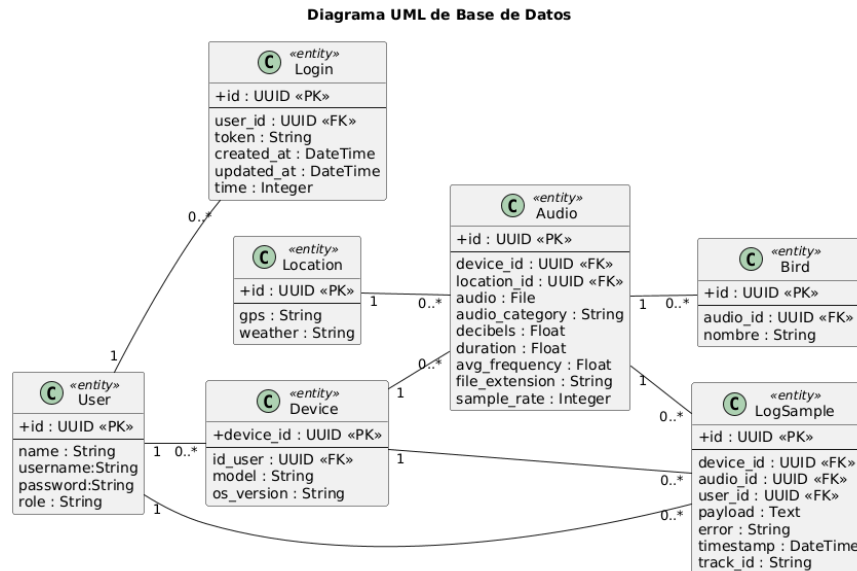


Figura 2. Diagrama de la base de datos

El modelo contempla las siguientes entidades principales: **User** (usuarios del sistema con roles diferenciados), **Device** (dispositivos de captura asociados a un usuario), **audio** (registros de audio con sus metadatos

acústicos y de clasificación), **Location** (coordenadas GPS y condiciones climáticas), **Login** (sesiones de autenticación con tokens), **bird**(Los pájaros identificados) y **log_sample** (registro de eventos y errores del sistema para auditoría).

El diccionario de datos se encuentra en los Anexos (Sección ??).

4.3. Modelo Físico (Infraestructura de Despliegue)

El modelo físico que utilizará el sistema que soluciona la problemática indicada, corresponde a una arquitectura distribuida tipo cliente-servidor, compuesta por múltiples dispositivos de captura de audio y un servidor central. Los dispositivos, en este caso dispositivos móviles ubicados en distintas zonas de la universidad, actúan como nodos que registran el audio del entorno y envían esta información a través de la red hacia el servidor.

El servidor central es el encargado de recibir la información obtenida para luego procesarlos para analizar las características del audio, hacer una clasificación y captar la detección de aves, y posteriormente, almacenar esta información en la base de datos. De esta manera, el sistema centraliza el procesamiento, y se realiza la captura de datos mediante diferentes dispositivos distribuidos en el entorno físico de la universidad.

En la figura 3, se muestra la distribución de la arquitectura del sistema de despliegue.

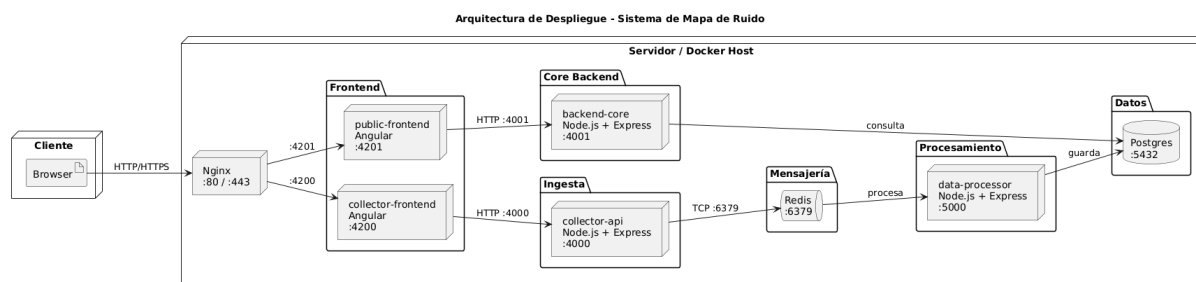


Figura 3. Diagrama de despliegue

4.4. Especificaciones mínimas

1. Servidor principales (frontend y backend)

- CPU: 8 vCPU
- RAM: 16 GB
- Disco: 200 GB SSD
- Red: mayor a 1 Gbps

Flask y Nginx tienen un consumo mínimo, Redis en memoria requiere que sea estable y para los Workers de audio existe alto consumo de CPU

2. Servidor de base de datos (PostgreSQL)

- CPU: 4 vCPU
- RAM: 16 GB
- Disco: 500 GB SSD (alto I/O)

Escrituras constantes desde procesamiento, consultas del frontend, indexación de datos acústicos

3. Nodo(s) de procesamiento (workers escalables)

- CPU: 4-8 vCPU
- RAM: 16-32 GB
- Disco: 100 GB

Procesamiento de alto rendimiento al procesar audios y utilizar modelos de IA para identificar aves.

4. Redis

- CPU: 2 vCPU
- RAM: 32 GB
- Disco: 20 GB

RAM dependiente del tamaño de las colas y la carga de trabajo

4.5. Modelo Fundamental

Modelo de Interacción: El sistema opera bajo un modelo *asíncrono* (SDA). La comunicación entre la collector-api y el data-processor es asíncrona mediante la cola Redis, lo que permite que la recepción de audio no bloquee al procesamiento. No se asume un reloj global; cada componente trabaja con timestamps locales sincronizados vía NTP. Los tiempos de transmisión de mensajes no están acotados, dado que el procesamiento de audio puede variar según la duración y complejidad del archivo. La comunicación entre frontends y backends es síncrona vía HTTP REST (request-response).

Modelo de Fallas: Se contemplan fallos por omisión en los canales de comunicación (mensajes perdidos entre componentes), que se mitigan mediante el mecanismo de reintentos y la persistencia de Redis. Se implementa logging centralizado en la tabla `log_sample` para detectar y diagnosticar fallos. El sistema enmascara fallos parciales del procesador mediante múltiples instancias del worker.

Modelo de Seguridad: Se implementa autenticación basada en tokens JWT, almacenados en la tabla Login. Los canales de comunicación externos se protegen mediante HTTPS/TLS. Se definen roles de usuario para controlar el acceso a los recursos. Las contraseñas se almacenan con hash bcrypt. En la Iteración 3 se incorporará cifrado de mensajes entre componentes internos.

En conjunto, este modelo permite diseñar un sistema robusto, capaz de operar en un entorno distribuido real, considerando las limitaciones de la red, la posibilidad de fallos y la necesidad de proteger la información.

5. Herramientas y Tecnologías

5.1. Tipo de Herramientas

La propuesta utiliza exclusivamente **herramientas de software libre**, lo cual se justifica por las siguientes razones: no genera costos de licenciamiento, además existe amplia documentación y comunidad de soporte para cada herramienta; permite modificar y adaptar los componentes según las necesidades del proyecto; y cumple con el requisito de aprovechar los recursos de manera eficiente sin gastos adicionales.

Como **ventajas**, se tiene el costo cero de licencias, la flexibilidad para personalizar componentes y la independencia de proveedores. Como **desventajas**, el soporte depende de la comunidad (sin SLA formal) y la configuración inicial puede requerir mayor esfuerzo técnico que soluciones comerciales preconfiguradas.

5.2. Bibliotecas y Frameworks de Python

- **Flask 3.0:** Framework web ligero para implementar las APIs REST (backend-core y collector-api). Se elige por su simplicidad, flexibilidad y amplia adopción en proyectos de microservicios.
- **librosa 0.10:** Biblioteca para análisis de audio. Se utiliza para extraer características acústicas como nivel de decibeles, frecuencia promedio, duración y espectrogramas.
- **SQLAlchemy 2.0:** ORM y toolkit SQL para Python. Permite mapear las tablas de PostgreSQL a clases Python, facilitando las consultas, inserciones y relaciones entre entidades sin escribir SQL directo.
- **redis-py 5.0:** Cliente Redis para Python. Se utiliza para encolar y consumir tareas de procesamiento de audio.
- **rq (Redis Queue) 1.16:** Biblioteca para gestión de colas de trabajo basada en Redis. Facilita la creación de workers que procesan tareas en segundo plano.
- **PyJWT 2.8:** Biblioteca para generación y validación de tokens JWT, utilizada en el sistema de autenticación.
- **Flask-CORS 4.0:** Extensión de Flask para habilitar CORS, necesaria para la comunicación con los frontends Angular.
- **birdnetlib 1.2.2:** Modelo de deep learning para la identificación de especies de aves a partir de audio. Permite clasificar audio y predecir especies presentes según coordenadas geográficas y semana del año, lo que posibilita filtrar las aves esperadas en la zona de Valdivia según la época de grabación.
- **YAMNet 1:** Modelo preentrenado de Google para clasificación de eventos sonoros. Reconoce 521 categorías de audio incluyendo tráfico, voces, lluvia, maquinaria, entre otros. Se utiliza como clasificador general complementario a BirdNET.
- **unicorn 21.2:** Servidor WSGI de producción para servir las aplicaciones Flask dentro de los contenedores Docker.

5.3. Presupuesto

Las herramientas utilizadas en el proyecto corresponden a software libre y modelos de uso académico sin costo de licenciamiento. Los modelos de clasificación BirdNET y YAMNet se distribuyen bajo licencias que permiten su uso con fines educativos y de investigación. La infraestructura de despliegue utiliza servidores disponibles en la universidad, por lo que no existe un presupuesto significativo asociado al desarrollo ni a la operación del sistema.

6. Preguntas al Cliente y Supuestos

1. **¿Qué sistema operativo utilizan los servidores?**
Se asume que todos los servidores del sistema operan sobre distribuciones Linux (por ejemplo, Ubuntu Server 22.04 LTS), utilizando contenedores Docker para la gestión y despliegue de los distintos servicios. Esto permite asegurar portabilidad, aislamiento y escalabilidad en la infraestructura.
2. **¿Cuántos usuarios concurrentes se esperan?**
Se estima una carga de aproximadamente 700 usuarios concurrentes en el sistema, considerando tanto usuarios que envían grabaciones como aquellos que consultan el mapa de ruido en tiempo real.
3. **¿Cuál es la duración promedio de las grabaciones de audio?**
Se define una duración promedio de entre 10 y 30 segundos por grabación de audio, lo cual permite equilibrar la calidad del análisis acústico con la eficiencia en el procesamiento y almacenamiento.
4. **¿En qué formato se capturan los archivos de audio?**
Los archivos de audio serán capturados y almacenados en formato `.mp3`, `.wav` o `.m4a`, debido a su buena relación entre calidad de sonido y tamaño de archivo, lo que facilita la transmisión y almacenamiento en el sistema distribuido.
5. **¿Qué categorías de sonido se desean clasificar?**
El sistema estará orientado principalmente a la clasificación de sonidos relacionados con la contaminación acústica (por ejemplo, ruidos urbanos) y sonidos de aves, permitiendo así el análisis tanto de impacto ambiental como de biodiversidad.
6. **¿Puede existir pérdida de datos durante la transmisión?**
Se asume que pueden ocurrir pérdidas de datos debido a fallos de red, desconexiones de dispositivos o errores en los servicios. Para mitigar este riesgo, el sistema implementa mecanismos de reintento en el envío de datos, validación de integridad de archivos y el uso de colas persistentes en Redis. Además, se contempla el almacenamiento temporal en el dispositivo cliente en caso de fallos de conectividad, permitiendo reenviar los datos posteriormente.
7. **¿Cómo se espera que los usuarios interactúen con el sistema y que necesidades de UX son importantes?**
Se espera que los usuarios interactúen con el sistema a través de una interfaz web simple y intuitiva, que les permita subir audios del entorno y visualizar el mapa de ruido generado. La interacción debe ser rápida y de bajo esfuerzo, considerando que el proceso principal es capturar el audio y enviarlo al sistema. Es fundamental que exista retroalimentación hacia el usuario en cada etapa, así como la confirmación del envío, estado del procesamiento y visualización de resultados.
8. **¿Qué requisitos técnicos necesito para usar la app?**
Para utilizar la aplicación, los usuarios requieren de un dispositivo con conexión a internet que cuente con un navegador web. Además, es necesario que el dispositivo cuente con capacidad de capturar audio mediante el micrófono, ya sea integrado o externo.
9. **¿Qué tipo de visualización se espera (mapas de calor, gráficos temporales, reportes, etc.)?**
Se mostrará un mapa de calor (heatmap) que represente la intensidad sonora en decibeles por zona, y una vista de clasificación que permita filtrar los eventos acústicos por categoría (tráfico, aves, actividad humana, etc.) según lo que el usuario desee consultar.
10. **¿Qué nivel de precisión se espera en la clasificación de sonidos (por ejemplo, identificación de aves)?**
Se espera que el sistema sea capaz de detectar y clasificar diferentes tipos de ruido incluyendo fuentes como personas, animales, vehículos, etc., alcanzando un nivel de precisión de al menos un 75 %.

7. Mejoras y Cambios

7.1. Base de datos

- Se modificó el identificador primario de la tabla **LOGIN**, pasando de **UUID**.
- Se modificó el identificador primario de la tabla **BIRDS**, pasando de **UUID**.
- Se modificó el tipo del campo **time** de la tabla **LOGIN**, pasando de **TIMESTAMP** a **VARCHAR**
- Se reemplazó el campo **gps** de la tabla **LOCATIONS** por los campos **latitude** y **longitude**, ambos de tipo **double precision**, permitiendo representar la ubicación geográfica de forma más estructurada.
- Se cambió el campo **audio** de la tabla **AUDIOS** por **audio_file**, definido como **bytea**, permitiendo almacenar el archivo de audio directamente en la base de datos.
- Se ajustó la relación entre **AUDIOS** y **LOCATIONS**, utilizando el campo **location** como clave foránea hacia **LOCATIONS(id)**.
- Se modificó la tabla **log_sample**, reemplazando algunos nombres de campos del modelo anterior: **audio_id**, **user_id**, **device_id** y **track_id** fueron adaptados a **id_audio**, **id_user**, **id_device** y **track**.
- Se agregó el campo **timestamp** en la tabla **log_sample**, con valor por defecto **now()**, para registrar automáticamente el momento en que se genera cada muestra de log.
- Se agregó el campo **payload** en la tabla **log_sample**, permitiendo almacenar información adicional asociada al procesamiento o envío de datos.

7.2. Bibliotecas y Frameworks

- Se actualizó la biblioteca **librosa** desde la versión 0.10 a la versión 0.11.0, incorporando mejoras de compatibilidad y procesamiento de audio.
- Se incorporó **birdnet-analyzer 2.4.0** reemplazando el análisis de audio de **birdnetlib 1.2.2**, basado en BirdNET para la detección e identificación de una mayor cantidad de especies de aves junto con una mayor precisión.
- Se incorporó **TensorFlow 2.16.2**, necesario para utilizar los **BirdNET**, como framework de aprendizaje profundo para la ejecución de modelos de clasificación de audio.
- Se incorporó **TensorFlow Hub 0.16.1** para la carga y utilización de modelos preentrenados como **BirdNET**.
- Se añadieron las bibliotecas **numpy**, **resampy** y **soundfile** como dependencias necesarias para el procesamiento y manipulación de señales de audio.
- Se incorporó **pytest 8.3.2** para la implementación de pruebas automatizadas del sistema.